



# JSigntPdf User Guide

Josef (kwart) Cacek

3.0.1

# Table of Contents

|                                                  |    |
|--------------------------------------------------|----|
| Introduction .....                               | 1  |
| License .....                                    | 1  |
| History .....                                    | 1  |
| Author .....                                     | 1  |
| Getting support .....                            | 1  |
| Installation and prerequisites .....             | 2  |
| Downloading JSigPdf .....                        | 2  |
| Java .....                                       | 2  |
| Obtaining a keystore .....                       | 2  |
| Demo files .....                                 | 3  |
| Launching JSigPdf .....                          | 4  |
| From the Windows installer or portable ZIP ..... | 4  |
| From the platform-independent ZIP .....          | 4  |
| Using the JavaFX UI .....                        | 5  |
| Opening a PDF document .....                     | 5  |
| Configuring signature options .....              | 6  |
| Placing a visible signature .....                | 7  |
| Signing the document .....                       | 8  |
| Using presets .....                              | 9  |
| Adjusting advanced preferences .....             | 10 |
| Classic Swing UI .....                           | 11 |
| Command line (batch mode) .....                  | 12 |
| General options .....                            | 12 |
| Keystore and key options .....                   | 13 |
| Signature options .....                          | 13 |
| Output options .....                             | 13 |
| Visible signature options .....                  | 14 |
| Encryption and rights options .....              | 14 |
| TSA and certificate validation options .....     | 15 |
| Proxy options .....                              | 16 |
| Program exit codes .....                         | 16 |
| Examples .....                                   | 16 |
| Reading passwords from standard input .....      | 17 |
| Signing options .....                            | 19 |
| Keystore selection .....                         | 19 |
| Key alias & key password .....                   | 19 |
| Signing properties .....                         | 19 |
| Encryption .....                                 | 20 |

|                                                     |    |
|-----------------------------------------------------|----|
| Visible signature options .....                     | 21 |
| TSA — timestamps .....                              | 22 |
| Certificate revocation checking .....               | 23 |
| Proxy settings .....                                | 23 |
| Using hardware tokens for signing .....             | 24 |
| Advanced application configuration .....            | 25 |
| Migration from earlier versions .....               | 25 |
| Configuration directory .....                       | 25 |
| Java VM options for the Windows EXE launchers ..... | 26 |
| Troubleshooting .....                               | 28 |
| Out of memory error .....                           | 28 |
| SSL/TLS certificate errors .....                    | 28 |
| Certificate not found in keystore .....             | 28 |
| Timestamp server unreachable .....                  | 28 |
| Other command line tools .....                      | 29 |
| InstallCert Tool .....                              | 29 |

# Introduction

JSigPdf is an open-source application that adds digital signatures to PDF documents. It's written in the Java programming language and can be launched on most current operating systems. Users can control the application using a modern JavaFX graphical interface, the classic Swing GUI, or command line arguments. Main features:

- supports visible signatures
- can set certification level
- supports PDF encryption with setting rights
- timestamp support
- certificate revocation checking (CRL and/or OCSP)

## License

JSigPdf is released under LGPL and/or MPL license. It can be freely used for both personal and commercial use. For details look directly to the license files.

## History

The project started at the beginning of 2008.

A greater change comes in 2021, where the project was switched to use the OpenPDF library instead of the old version of the iText library.

Starting with version 3.0.0, JSigPdf ships with a new document-centric JavaFX interface as the default graphical frontend. The classic Swing GUI remains available for backward compatibility.

## Author

The author of JSigPdf is Czech developer Josef (kwart) Cacek. He works in Java since 2000. Some links to Josef's projects:

- <https://github.com/intoolswetrust/>
- <https://github.com/kwart/>

## Getting support

If you don't find the relevant information in this document or on the JSigPdf GitHub page (<https://github.com/intoolswetrust/jsignpdf>), use the JSigPdf Google Group to ask the community.

[JSigPdf Google Group](#)

# Installation and prerequisites

## Downloading JSignPdf

JSignPdf releases are available on the GitHub releases page:

<https://github.com/intoolswetrust/jsignpdf/releases>

Each release provides the following artifacts:

| Artifact                                      | Description                                                                                                                                                                                                                                         |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>jsignpdf-\${VERSION}.zip</code>         | Platform-independent ZIP archive containing the JAR files ( <code>JSignPdf.jar</code> , <code>InstallCert.jar</code> ), a shell launcher for Linux/macOS, the documentation and the demo material. Works on any OS with Java 21 or newer installed. |
| <code>JSignPdf-\${VERSION}-win-x64.exe</code> | Windows 64-bit EXE installer. Bundles its own Java runtime, installs to <i>Program Files</i> by default, creates Start Menu shortcuts and registers an Add/Remove Programs entry.                                                                   |
| <code>JSignPdf-\${VERSION}-win-x64.msi</code> | Windows 64-bit MSI installer. Functionally equivalent to the EXE installer; preferred for deployment via group policy or enterprise software-distribution tools.                                                                                    |
| <code>JSignPdf-\${VERSION}-win-x64.zip</code> | Windows 64-bit portable ZIP. Contains the fully packaged application (including a bundled JRE); just unzip and run <code>JSignPdf.exe</code> .                                                                                                      |



32-bit Windows is no longer supported by the dedicated Windows installers. Users on 32-bit Windows can still run JSignPdf from the platform-independent ZIP with their own 32-bit JRE.

## Java

If you use the platform-independent ZIP archive, you need Java Runtime Environment (JRE) version 21 or newer. You can download it freely from the web, for instance:

- [Eclipse Adoptium](#)
- [Azul Zulu](#)



The Windows EXE / MSI / portable ZIP variants all bundle their own JRE, so a separate Java installation is not required on Windows.

## Obtaining a keystore

To sign PDF documents you need a keystore containing your private key. If you don't have one yet:

- **For testing**, you can generate a self-signed certificate with the `keytool` utility that ships with Java:

```
keytool -genkeypair -alias mykey -keyalg RSA -keysize 2048 -keystore keystore.p12 -storetype PKCS12
```

- **For production use**, obtain a certificate from a trusted Certificate Authority (CA). The CA will typically issue a PKCS#12 (.p12 / .pfx) file.

The most common keystore types supported by Java are:

- PKCS#12 — keys stored in .p12 and .pfx files
- PKCS#11 — keys stored usually on hardware modules (see [Using hardware tokens for signing](#))
- JKS (Java Key Store)
- WINDOWS-MY — supported on MS Windows. You can use directly your certificates imported into your system.

JSigPdf has been also extended to support external keystore types like smart cards, or network HSMs. The first example is CloudFoxy (<https://gitlab.com/cloudfoxy>).

## Demo files

Every release bundles a small demo/ folder so you can try JSigPdf immediately without generating your own certificate or finding a sample PDF:

| File                  | Description                                                                                                                                                           |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| service-agreement.pdf | Sample unsigned PDF document.                                                                                                                                         |
| jsmith.p12            | Demo PKCS#12 keystore with a self-signed certificate for CN=John Smith. The keystore and key password is 123456.                                                      |
| README.md             | Walk-through of the most common command-line scenarios (basic signature, visible signature, timestamped signature with FreeTSA, appending a signature, listing keys). |

You can find the folder:

- at the top level of the platform-independent ZIP (jsignpdf-\${VERSION}.zip)
- under <install dir>\app\demo\ after installing with the Windows EXE / MSI installer
- under JSigPdf\app\demo\ in the Windows portable ZIP (JSigPdf-\${VERSION}-win-x64.zip)



The bundled keystore is meant purely for experimentation. Do not use it to sign real documents.

# Launching JSignPdf

## From the Windows installer or portable ZIP

When JSignPdf is installed via the Windows EXE / MSI installer (or extracted from the [win-x64.zip](#) portable archive), four launchers are available:

| Launcher                           | Purpose                                                                                                                                                 |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">JSignPdf.exe</a>       | Starts the JavaFX graphical interface (the default). Also available as a Start Menu shortcut after installation.                                        |
| <a href="#">JSignPdf-swing.exe</a> | Starts the classic Swing graphical interface (equivalent to passing <a href="#">-Djsignpdf.swing=true</a> on the command line).                         |
| <a href="#">JSignPdfC.exe</a>      | Console-mode launcher for the signer. Use this in batch scripts, CI pipelines or whenever you want the output to appear in the current terminal window. |
| <a href="#">InstallCert.exe</a>    | Command-line tool for importing a server certificate into the bundled JRE's <a href="#">cacerts</a> truststore (see <a href="#">InstallCert Tool</a> ). |

All four launchers share the bundled JRE — you do **not** need to have Java installed separately. For installer-based deployments the launchers live in `<install dir>\` (with Start Menu shortcuts to the GUI launchers); in the portable ZIP they are directly in the extracted folder.

## From the platform-independent ZIP

All platforms (with Java 21 or newer installed) can run the JAR file *JSignPdf.jar* directly. Use the following command in the directory where the application is located:

```
$ java -jar JSignPdf.jar
```

On Linux and macOS a convenience shell script is bundled in [bin/jsignpdf.sh](#) which adds the JVM options required on Java 21+ and then launches the JAR.

By default, JSignPdf opens the JavaFX graphical interface (see [Using the JavaFX UI](#)). To start the classic Swing GUI instead, set the [jsignpdf.swing](#) system property:

```
$ java -Djsignpdf.swing=true -jar JSignPdf.jar
```

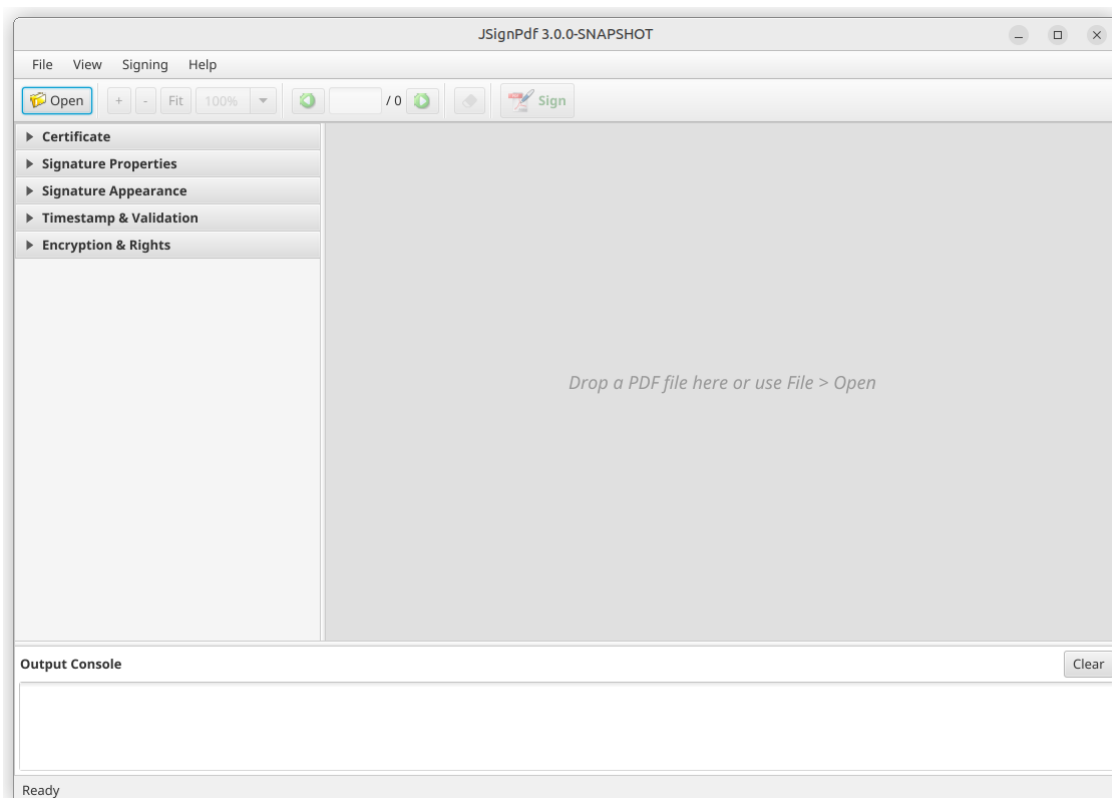
If command line arguments for signing are provided, JSignPdf runs in batch mode without opening any GUI (see [Command line \(batch mode\)](#)).

# Using the JavaFX UI

The JavaFX interface is the default graphical frontend for JSignPdf. It uses a document-centric workflow: you open a single PDF, configure the signature, and sign it. For signing multiple files in one go, use the [command line](#).

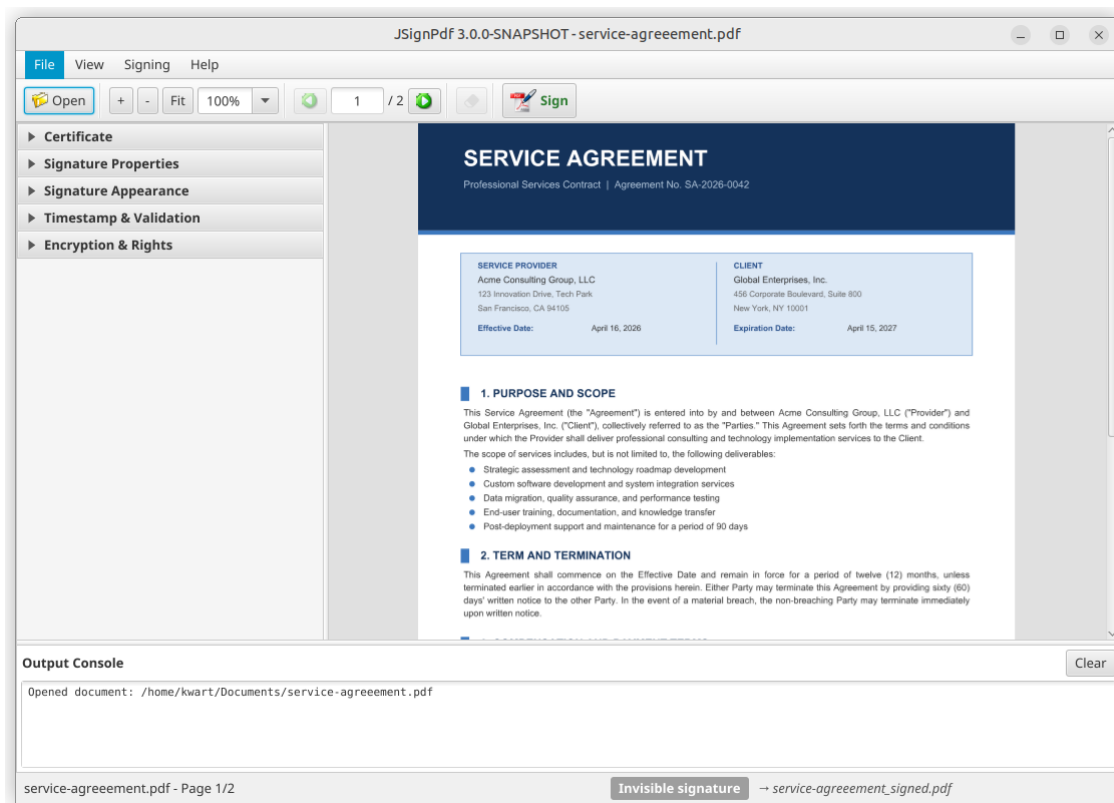
## Opening a PDF document

When JSignPdf starts, the main window is empty and invites you to load a document. You can either drag and drop a PDF file onto the window or use *File > Open* (or the *Open* button in the toolbar).



Once a PDF is loaded, the document preview appears on the right, and the configuration panels on the left become active. The status bar at the bottom shows the file name and page count.

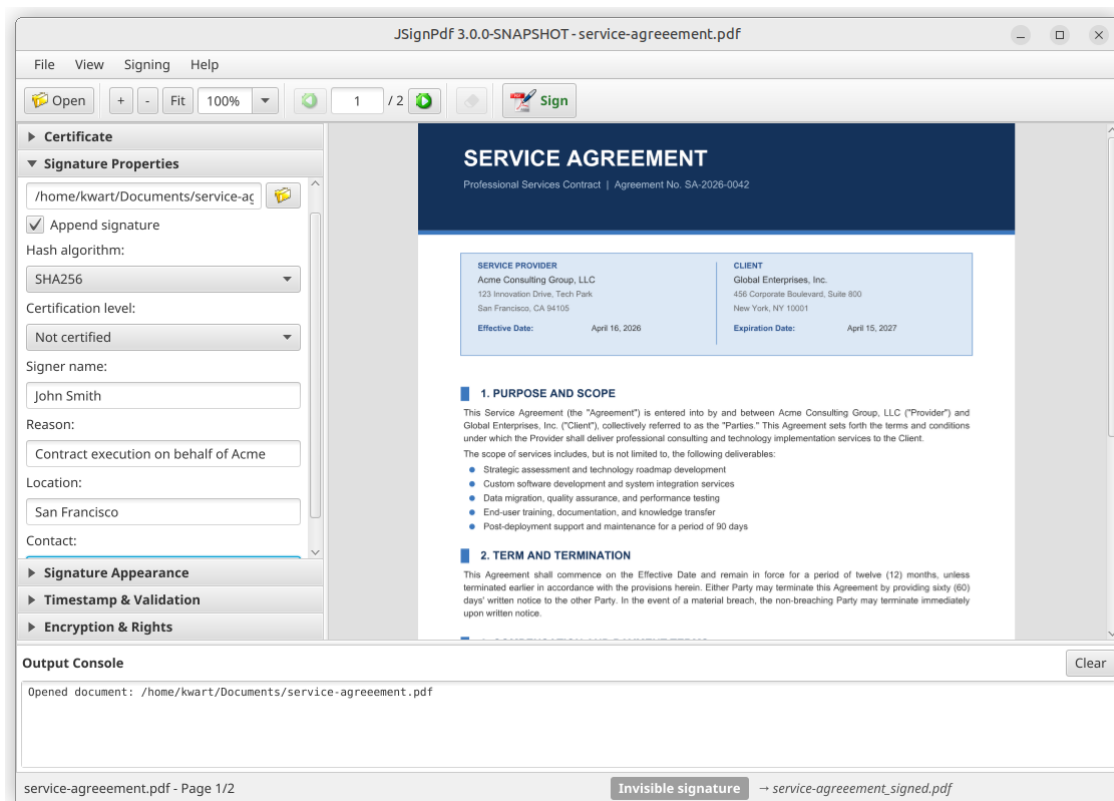




## Configuring signature options

The left sidebar contains five collapsible panels that expose all signing options. Expand a panel by clicking its header.

- **Certificate** — keystore type, keystore file & password, key alias & password (see [Keystore selection](#) and [Key alias & key password](#))
- **Signature Properties** — hash algorithm, certification level, append mode, reason / location / contact, input & output file paths (see [Signing properties](#))
- **Signature Appearance** — visible signature toggle, render mode, texts, images, font size (see [Visible signature options](#))
- **Timestamp & Validation** — TSA server, CRL, OCSP (see [TSA — timestamps](#) and [Certificate revocation checking](#))
- **Encryption & Rights** — encryption mode, passwords, certificate, document rights (see [Encryption](#))

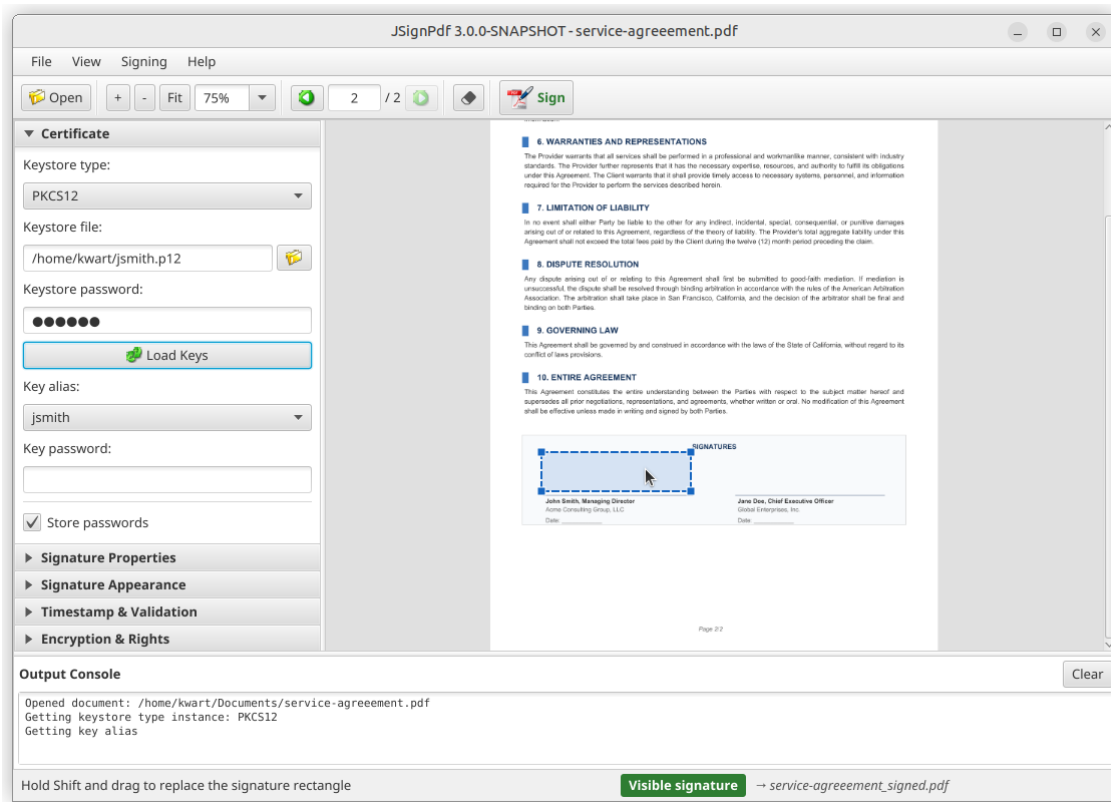


At a minimum, you need to select a keystore and provide its password in the *Certificate* panel. All other options have sensible defaults.

## Placing a visible signature

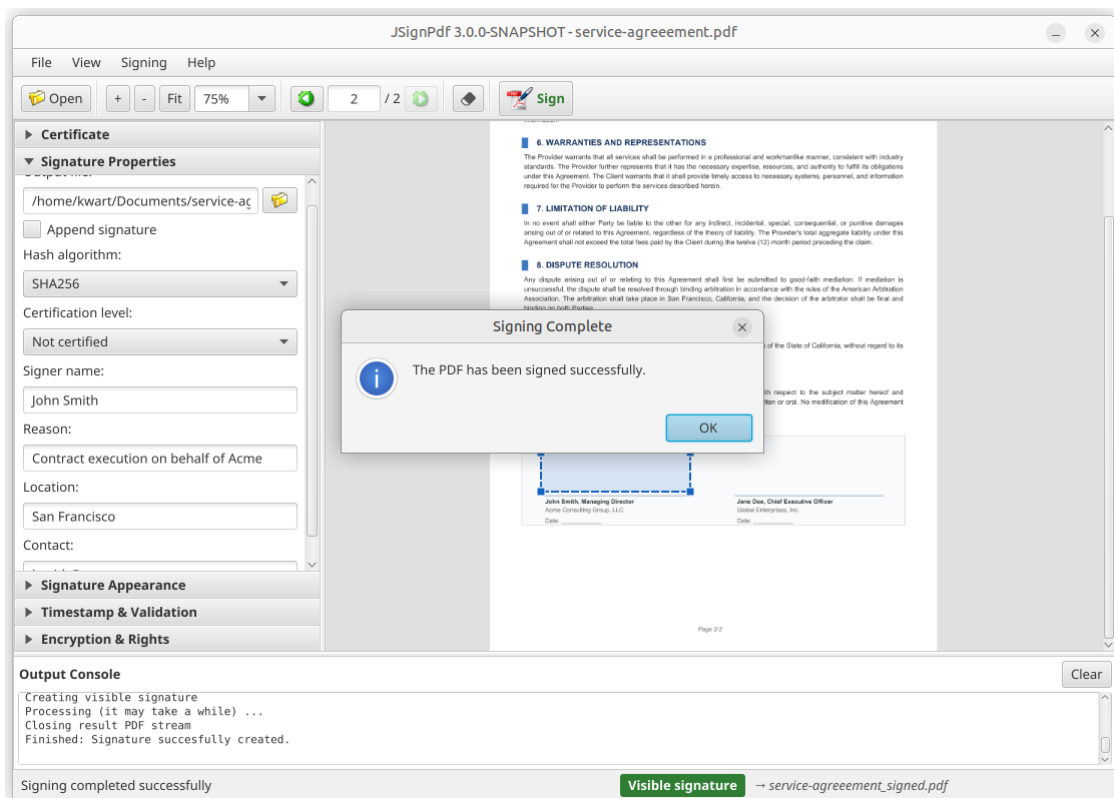
To add a visible signature to the PDF, enable *Visible signature* in the *Signature Appearance* panel. You can then configure the render mode, signature text, status text, images, and font size.

To position the signature on the page, hold **Shift** and drag a rectangle directly on the document preview. The coordinates are updated automatically in the sidebar.



## Signing the document

When you are ready, click the *Sign* button in the toolbar. JSigntool signs the document and displays the result in the Output Console at the bottom of the window.



The signed PDF is saved to the output path shown in the *Signature Properties* panel. By default, it is the input file name with a *\_signed* suffix (e.g. *document\_signed.pdf*).

# Using presets

A **preset** is a named bundle of signing settings that you can save once and reload with a single click. Presets are useful when you regularly switch between different configurations — for example, one with a timestamp authority enabled and one without, or different visible-signature layouts for different document types.

Presets capture everything in the [Signing options](#) reference (keystore, signature properties, visible-signature options, TSA, encryption and rights, OCSP/CRL, proxy). Passwords are included only if *Store passwords* is enabled at the time you save the preset; in that case they are encrypted with a machine-local seed tied to the current user account and are silently ignored on load if the preset file is used under a different `user.home`. If you want a preset without stored passwords, disable *Store passwords* before saving — no password keys are written to the file in that case. Loading a preset without passwords also clears any passwords already entered in the UI, so switching to a password-less preset never silently reuses credentials from an earlier selection. The currently loaded input and output file paths are not part of a preset.

## Loading a preset

The *Preset* drop-down in the toolbar shows every saved preset, sorted by name. Picking an entry immediately loads its settings into the UI — no confirmation, no extra click. After the load, the drop-down returns to its *Load preset...* placeholder. There is no "currently active preset" tracked anywhere: the drop-down is just a launcher, and whatever you do after loading (edits, saving as a new preset, signing) is independent of the preset you came from.

If no presets exist yet, the drop-down is disabled and labelled *No presets saved*.

## Saving a new preset

Choose *Presets > Save current as new preset...* from the menu bar. Enter a name (up to 60 characters) and confirm. The preset appears in the toolbar drop-down immediately.

## Managing existing presets

Choose *Presets > Manage presets...* to open the management dialog. Each row offers three actions:

- **Rename** — change the display name. The underlying file is not renamed; the name is stored inside the file.
- **Overwrite** — replace the preset's stored settings with the current live settings. The preset's name and creation date are preserved. Asks for confirmation because the previous contents are lost.
- **Delete** — remove the preset file. Asks for confirmation.

Presets are plain Java properties files under the [configuration directory](#).

# Adjusting advanced preferences

Open *File > Preferences...* (shortcut **Ctrl+,** / **⌘,**) for application-wide tweaks that fall outside the per-document signing options. The dialog has the following tabs:

- **Visible Signature** — font file, font name and encoding for the visible-signature L2 text
- **Certificates** — which certificate checks (validity, key usage, critical extensions) are applied when listing aliases
- **Network** — relax SSL security for TSA / OCSP / CRL traffic
- **PDF Render** — order and selection of preview backends (JPedal, Apache PDFBox, OpenPDF)
- **TSA** — default hash algorithm for timestamp requests
- **PKCS#11** — raw text of the SunPKCS11 provider configuration (see [Using hardware tokens for signing](#))

Most settings apply immediately on *OK*. The two tabs marked *Takes effect after restart* — Network and PKCS#11 — only take effect on the next launch. *Reset section to defaults* restores the bundled defaults for the active tab only; other tabs keep their current values.

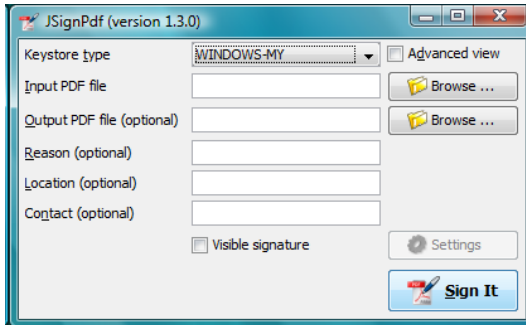
The values persist in `<config-dir>/advanced.properties` (see [Advanced application configuration](#)).

# Classic Swing UI

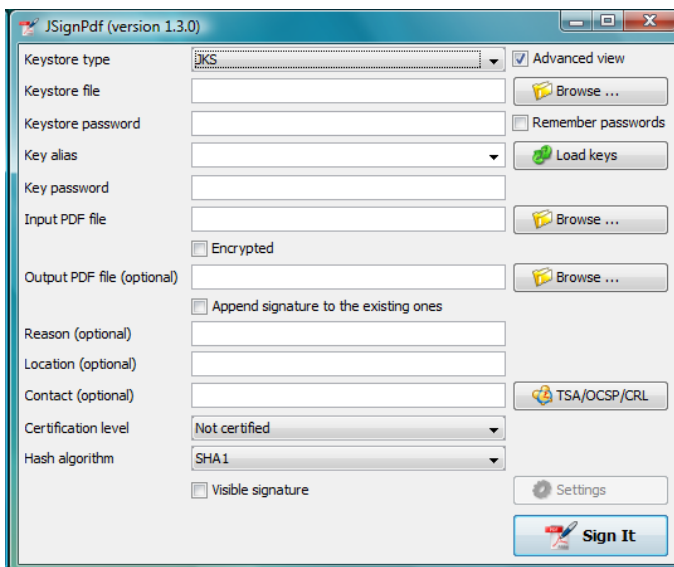
The classic Swing GUI is the original graphical interface from earlier versions of JSignPdf. It remains available for backward compatibility. To start it, set the `jsignpdf.swing` system property:

```
$ java -Djsignpdf.swing=true -jar JSignPdf.jar
```

The Swing GUI opens in a simplified view. Fill in the keystore type, keystore file and password, the input PDF file, and press *Sign It*.



Select the *Advanced view* checkbox to access additional options such as key alias, append signature, certification level, hash algorithm, encryption, visible signature, TSA, and certificate revocation settings.



All options available in the Swing GUI correspond to the options described in the [Signing options](#) chapter.

# Command line (batch mode)

JSignPdf can be used in batch mode to sign one or more PDF files from the command line. This is useful for scripting and for signing multiple files at once.

The options are grouped by category below. For detailed descriptions of each option, see the [Signing options](#) chapter.

```
usage: java -jar JSignPdf.jar [file1.pdf [file2.pdf ...]] [-a] [--bg-path
<file>] [--bg-scale <scale>] [-c <contact>] [-cl <level>] [--crl] [-d
<path>] [--disable-acrobat6-layer-mode] [--disable-assembly]
[--disable-copy] [--disable-fill] [--disable-modify-annotations]
[--disable-modify-content] [--disable-screen-readers] [-e] [-ec <file>]
[--enable-stdin-passwords] [-fs <size>] [--gui] [-h] [-ha <algorithm>]
[--img-path <file>] [-ka <alias>]
[-ki <index>] [-kp <password>] [-ksf <file>] [-ksp <password>] [-kst
<type>] [-l <location>] [--l2-text <text>] [--l4-text <text>] [-lk]
[-lkt] [-llx <position>] [-lly <position>] [-lp] [-lpf <file>] [--ocsp]
[--ocsp-server-url <responderUrl>] [-op <prefix>] [-opwd <password>] [-os
<suffix>] [-pe <mode>] [-pg <pageNumber>] [-pr <right>] [--proxy-host
<hostname>] [--proxy-port <port>] [--proxy-type <type>] [-q] [-r
<reason>] [--render-mode <mode>] [-sn <signer>] [-ta <method>] [-ts <URL>]
[--tsa-policy-oid <policyOID>] [-tscf <file>] [-tscp <password>] [-tsct
<ks-type>] [-tsh <algorithm>] [-tsp <password>] [-tsu <username>] [-upwd
<password>] [-urx <position>] [-ury <position>] [-v] [-V]
```

## General options

| Option                                           | Description                                                                                                                                                                                                                           |
|--------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>-h, --help</b>                                | Prints the help screen.                                                                                                                                                                                                               |
| <b>-v, --version</b>                             | Shows the application version.                                                                                                                                                                                                        |
| <b>-q, --quiet</b>                               | Quiet mode — suppresses info messages during signing.                                                                                                                                                                                 |
| <b>--gui</b>                                     | (Beta) Show the GUI even when other command line arguments are provided.                                                                                                                                                              |
| <b>-lkt, --list-keystore-types</b>               | Lists available keystore types (use as values for <b>-kst</b> ).                                                                                                                                                                      |
| <b>-lk, --list-keys</b>                          | Lists keys in the chosen keystore.                                                                                                                                                                                                    |
| <b>-lp, --load-properties</b>                    | Loads properties from the default file (created by the GUI application).                                                                                                                                                              |
| <b>-lpf, --load-properties-file &lt;file&gt;</b> | Loads properties from the given file. The file can be created by copying the default <b>config.properties</b> from the <a href="#">Configuration directory</a> .                                                                      |
| <b>--enable-stdin-passwords</b>                  | Allow reading password values from standard input. When set, a password option value of <b>-</b> means "read one line from stdin" (or prompt, on an interactive console). See <a href="#">Reading passwords from standard input</a> . |

# Keystore and key options

See [Keystore selection](#) and [Key alias & key password](#) for details.

| Option                                                  | Description                                                                                                                                                           |
|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>-kst, --keystore-type &lt;type&gt;</code>         | Sets the keystore type. Use <code>-lkt</code> to list available types.                                                                                                |
| <code>-ksf, --keystore-file &lt;file&gt;</code>         | Path to the keystore file ( <code>.p12</code> , <code>.pfx</code> , <code>.jks</code> , ...). Not needed for keystores without files (e.g. <code>WINDOWS-MY</code> ). |
| <code>-ksp, --keystore-password &lt;password&gt;</code> | Password for the keystore. Use <code>-</code> to read from stdin (requires <code>--enable-stdin-passwords</code> ).                                                   |
| <code>-ka, --key-alias &lt;alias&gt;</code>             | Name (alias) of the key to use for signing. If not given, the first key in the keystore is used. Use <code>-lk</code> to list available aliases.                      |
| <code>-ki, --key-index &lt;index&gt;</code>             | Zero-based index of the key to use. Lower priority than <code>-ka</code> .                                                                                            |
| <code>-kp, --key-password &lt;password&gt;</code>       | Password for the individual key (if different from the keystore password). Use <code>-</code> to read from stdin (requires <code>--enable-stdin-passwords</code> ).   |

## Signature options

See [Signing properties](#) for details.

| Option                                                | Description                                                                                                                                                                                                                             |
|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>-a, --append</code>                             | Append signature to existing ones instead of replacing them.                                                                                                                                                                            |
| <code>-ha, --hash-algorithm &lt;algorithm&gt;</code>  | Hash algorithm for the signature. Default: <code>SHA1</code> . Values: <code>SHA1</code> , <code>SHA256</code> , <code>SHA384</code> , <code>SHA512</code> , <code>RIPEMD160</code> .                                                   |
| <code>-cl, --certification-level &lt;level&gt;</code> | Certification level. Default: <code>NOT_CERTIFIED</code> . Values: <code>NOT_CERTIFIED</code> , <code>CERTIFIED_NO_CHANGES_ALLOWED</code> , <code>CERTIFIED_FORM_FILLING</code> , <code>CERTIFIED_FORM_FILLING_AND_ANNOTATIONS</code> . |
| <code>-sn, --signer-name &lt;signer&gt;</code>        | Signer name. Defaults to the common name (CN) of the chosen certificate.                                                                                                                                                                |
| <code>-r, --reason &lt;reason&gt;</code>              | Reason for the signature. Empty by default.                                                                                                                                                                                             |
| <code>-l, --location &lt;location&gt;</code>          | Location of the signature (e.g. Washington DC). Empty by default.                                                                                                                                                                       |
| <code>-c, --contact &lt;contact&gt;</code>            | Signer's contact details.                                                                                                                                                                                                               |

## Output options

| Option                                        | Description                                           |
|-----------------------------------------------|-------------------------------------------------------|
| <code>-d, --out-directory &lt;path&gt;</code> | Folder for signed documents. Default: current folder. |



| Option                                        | Description                                                                                                                                   |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>-op, --out-prefix &lt;prefix&gt;</code> | Prefix for the signed filename. Default: empty.                                                                                               |
| <code>-os, --out-suffix &lt;suffix&gt;</code> | Suffix for the signed filename. Default: <code>_signed</code> (e.g. <code>mydocument.pdf</code> becomes <code>mydocument_signed.pdf</code> ). |

## Visible signature options

See [Visible signature options](#) for details.

| Option                                      | Description                                                                                                                                                                                                   |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>-V, --visible-signature</code>        | Enables a visible signature.                                                                                                                                                                                  |
| <code>-pg, --page &lt;pageNumber&gt;</code> | Page for the visible signature. Default: <code>1</code> . If out of bounds, the last page is used.                                                                                                            |
| <code>-llx &lt;position&gt;</code>          | Lower left corner X-axis position (negative value is subtracted from page width).                                                                                                                             |
| <code>-lly &lt;position&gt;</code>          | Lower left corner Y-axis position (negative value is subtracted from page height).                                                                                                                            |
| <code>-urx &lt;position&gt;</code>          | Upper right corner X-axis position (negative value is subtracted from page width).                                                                                                                            |
| <code>-ury &lt;position&gt;</code>          | Upper right corner Y-axis position (negative value is subtracted from page height).                                                                                                                           |
| <code>--render-mode &lt;mode&gt;</code>     | Render mode. Default: <code>DESCRIPTION_ONLY</code> . Values: <code>DESCRIPTION_ONLY</code> , <code>GRAPHIC_AND_DESCRIPTION</code> , <code>SIGNATURE_AND_DESCRIPTION</code> .                                 |
| <code>--l2-text &lt;text&gt;</code>         | Signature text. Supports placeholders: <code>\${signer}</code> , <code>\${certificate}</code> , <code>\${timestamp}</code> , <code>\${location}</code> , <code>\${reason}</code> , <code>\${contact}</code> . |
| <code>--l4-text &lt;text&gt;</code>         | Status text.                                                                                                                                                                                                  |
| <code>-fs, --font-size &lt;size&gt;</code>  | Font size for signature text. Default: <code>10.0</code> .                                                                                                                                                    |
| <code>--img-path &lt;file&gt;</code>        | Image path for the visible signature.                                                                                                                                                                         |
| <code>--bg-path &lt;file&gt;</code>         | Background image path for the visible signature.                                                                                                                                                              |
| <code>--bg-scale &lt;scale&gt;</code>       | Background image scale. Positive: multiply size. Zero: stretch to fill. Negative: best-fit resize.                                                                                                            |
| <code>--disable-acrobat6-layer-mode</code>  | Disable Acrobat 6 layer mode (creates all signature layers instead of just n2 and n4).                                                                                                                        |

## Encryption and rights options

See [Encryption](#) for details.

| Option                                                  | Description                                                                                                                                                                                                                                 |
|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>-pe, --encryption &lt;mode&gt;</code>             | Encryption mode. Default: <b>NONE</b> . Values: <b>NONE</b> , <b>PASSWORD</b> , <b>CERTIFICATE</b> . Use together with <code>-upwd</code> and <code>-opwd</code> for <b>PASSWORD</b> mode, or <code>-ec</code> for <b>CERTIFICATE</b> mode. |
| <code>-e, --encrypted</code>                            | <i>Deprecated.</i> Use <code>-pe PASSWORD</code> instead.                                                                                                                                                                                   |
| <code>-opwd, --owner-password &lt;password&gt;</code>   | Owner password for encrypted documents. Use <code>-</code> to read from stdin (requires <code>--enable-stdin-passwords</code> ).                                                                                                            |
| <code>-upwd, --user-password &lt;password&gt;</code>    | User password for encrypted documents. Use <code>-</code> to read from stdin (requires <code>--enable-stdin-passwords</code> ).                                                                                                             |
| <code>-ec, --encryption-certificate &lt;file&gt;</code> | Path to certificate file for <b>CERTIFICATE</b> encryption.                                                                                                                                                                                 |
| <code>-pr, --print-right &lt;right&gt;</code>           | Printing rights. Default: <b>ALLOW_PRINTING</b> . Values: <b>DISALLOW_PRINTING</b> , <b>ALLOW_DEGRADED_PRINTING</b> , <b>ALLOW_PRINTING</b> .                                                                                               |
| <code>--disable-copy</code>                             | Deny copy in encrypted documents.                                                                                                                                                                                                           |
| <code>--disable-assembly</code>                         | Deny assembly in encrypted documents.                                                                                                                                                                                                       |
| <code>--disable-fill</code>                             | Deny fill in encrypted documents.                                                                                                                                                                                                           |
| <code>--disable-modify-annotations</code>               | Deny modifying annotations in encrypted documents.                                                                                                                                                                                          |
| <code>--disable-modify-content</code>                   | Deny modifying content in encrypted documents.                                                                                                                                                                                              |
| <code>--disable-screen-readers</code>                   | Deny screen readers in encrypted documents.                                                                                                                                                                                                 |

## TSA and certificate validation options

See [TSA — timestamps](#) and [Certificate revocation checking](#) for details.

| Option                                                   | Description                                                                                                                                 |
|----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <code>-ts, --tsa-server-url &lt;URL&gt;</code>           | Address of the timestamping server (TSA). Enables timestamping.                                                                             |
| <code>-ta, --tsa-authentication &lt;method&gt;</code>    | TSA authentication method. Default: <b>NONE</b> . Values: <b>NONE</b> , <b>PASSWORD</b> , <b>CERTIFICATE</b> .                              |
| <code>-tsu, --tsa-user &lt;username&gt;</code>           | TSA username (for <b>PASSWORD</b> authentication).                                                                                          |
| <code>-tsp, --tsa-password &lt;password&gt;</code>       | TSA password (for <b>PASSWORD</b> authentication). Use <code>-</code> to read from stdin (requires <code>--enable-stdin-passwords</code> ). |
| <code>-tscf, --tsa-cert-file &lt;file&gt;</code>         | Path to keystore file with private key for TSA <b>CERTIFICATE</b> authentication.                                                           |
| <code>-tscp, --tsa-cert-password &lt;password&gt;</code> | Password for the TSA certificate keystore. Use <code>-</code> to read from stdin (requires <code>--enable-stdin-passwords</code> ).         |
| <code>-tsct, --tsa-cert-file-type &lt;ks-type&gt;</code> | Keystore type for TSA certificate authentication. Default: <b>PKCS12</b> .                                                                  |

| Option                                                         | Description                                                                      |
|----------------------------------------------------------------|----------------------------------------------------------------------------------|
| <code>-tsh, --tsa-hash<br/>-algorithm &lt;algorithm&gt;</code> | Hash algorithm for TSA queries. Default: <b>SHA-1</b> .                          |
| <code>--tsa-policy-oid<br/>&lt;policyOID&gt;</code>            | TSA policy OID to include in the timestamp request.                              |
| <code>--ocsp</code>                                            | Enable OCSP certificate validation.                                              |
| <code>--ocsp-server-url<br/>&lt;responderUrl&gt;</code>        | Default OCSP server URL (used when the signing certificate doesn't contain one). |
| <code>--crl</code>                                             | Enable CRL certificate validation.                                               |

## Proxy options

See [Proxy settings](#) for details.

| Option                                         | Description                                                                               |
|------------------------------------------------|-------------------------------------------------------------------------------------------|
| <code>--proxy-type &lt;type&gt;</code>         | Proxy type. Default: <b>DIRECT</b> . Values: <b>DIRECT</b> , <b>HTTP</b> , <b>SOCKS</b> . |
| <code>--proxy-host<br/>&lt;hostname&gt;</code> | Hostname or IP address of the proxy server.                                               |
| <code>--proxy-port &lt;port&gt;</code>         | Port of the proxy server. Default: <b>80</b> .                                            |

## Program exit codes

| Code | Meaning                                                     |
|------|-------------------------------------------------------------|
| 0    | Program finished without errors.                            |
| 1    | Command line is in a wrong format.                          |
| 2    | No operation requested — e.g. no file for signing provided. |
| 3    | Signing of some, but not all, files failed.                 |
| 4    | Signing of all files failed.                                |
| 5    | Input file cannot be read.                                  |
| 6    | Common error.                                               |

## Examples

Sign a PDF using the Windows certificate store:

```
$ java -jar JSignPdf.jar -kst WINDOWS-MY mydocument.pdf
```

Sign and encrypt a PDF, disabling printing:

```
$ java -jar JSignPdf.jar -kst PKCS12 -ksf my_certificate.pfx \
```

```
-ksp myPrivateKeystorePassword -ka cert23 \  
-pe PASSWORD -opwd xxx123 -upwd 123xxx \  
-pr DISALLOW_PRINTING mydocument.pdf
```

List available keystore types:

```
$ java -jar JSigndPdf.jar -lkt
```

List key aliases in a keystore (quiet mode):

```
$ java -jar JSigndPdf.jar -kst PKCS12 -ksf my_certificate.pfx \  
-ksp myVeryPrivatePassword -lk -q
```

## Reading passwords from standard input

Passing a password on the command line exposes it to every local process that can read `/proc/<pid>/cmdline` (and to shell history). To avoid that, JSigndPdf can read password values from standard input (or, in an interactive terminal, from the console) instead.

Opt in with `--enable-stdin-passwords` and use `-` as the value of any password option. Any of the six password options can use the sentinel: `-ksp`, `-kp`, `-opwd`, `-upwd`, `-tscp`, `-tsp`.

*Single password, piped from the environment*

```
$ printf '%s\n' "$KS_PASSWORD" \  
| java -jar JSigndPdf.jar --enable-stdin-passwords \  
-kst PKCS12 -ksf keystore.p12 -ksp - -ka mykey \  
-d out/ input.pdf
```

When more than one option uses `-`, the values are consumed from stdin in a **fixed canonical order** regardless of the order in which they appear on the command line:

1. `--keystore-password (-ksp)`
2. `--key-password (-kp)`
3. `--owner-password (-opwd)`
4. `--user-password (-upwd)`
5. `--tsa-cert-password (-tscp)`
6. `--tsa-password (-tsp)`

*Keystore password first, then TSA password — even though the CLI lists them in the other order*

```
$ { printf '%s\n' "$KS_PASSWORD"; printf '%s\n' "$TSA_PASSWORD"; } \  
| java -jar JSigndPdf.jar --enable-stdin-passwords \  
-kst PKCS12 -ksf keystore.p12 -ksp - -ka mykey \  
-tsp -
```

```
-ts https://tsa.example/ -tsu tsauser -tsp - \  
-d out/ input.pdf
```

Before blocking on each read, JSignPdf prints a progress line such as `[jsignpdf] Reading password for --keystore-password (1/2) from stdin...` to `stderr` so you can verify the order matches what your pipe is feeding. Use `-q` / `--quiet` to suppress it.

When a console is attached (an interactive terminal), `-` switches to a prompted, no-echo read via `java.io.Console.readPassword`:

```
$ java -jar JSignPdf.jar --enable-stdin-passwords \  
-kst PKCS12 -ksf keystore.p12 -ksp - -ka mykey input.pdf  
Enter password for --keystore-password: ****
```



Without `--enable-stdin-passwords`, a literal `-` is still accepted as a password value (backwards compatible). JSignPdf will emit a one-time warning naming the flag, so a typo does not silently turn into a `-` password. The warning is suppressed by `-q`.



Only exactly `-` is the stdin sentinel—values such as `-abc` or `abc-` are treated as literal passwords. Note that Apache Commons CLI intercepts a bare `--` as the end-of-options marker before JSignPdf sees it, so `--` itself cannot be used as a literal password via `-ksp --`; this is a parser limitation.

# Signing options

This chapter describes all signing options in detail. The options are the same regardless of which interface you use (JavaFX GUI, Swing GUI, or command line). In the JavaFX interface the options are organized into sidebar panels; the corresponding panel name is noted in each section header.

## Keystore selection

*JavaFX panel: Certificate*

The Keystore means the location where the private keys are located.

By default, JSignPdf displays keystore types provided by Java Runtime itself and the Bouncy Castle cryptographic provider.

JSignPdf has been extended to support remote/external keystore types. The first entry is "CloudFoxy" (<https://gitlab.com/cloudfoxy>), which is a REST API for physical smart cards, initially developed to support eIDAS signatures.

If you use PKCS#12 or Java keystore types (JKS, JCEKS), you have to select the file where the keys are stored and provide the password of this file. Path to the keystore file can be inserted directly by typing or you can use the browse button to navigate through the file system with Open File Dialog.

## Key alias & key password

*JavaFX panel: Certificate*

When you have more private keys stored in the keystore, you can select which one will be used to sign the PDF file by filling the *Key alias* field. Either you can type alias name directly (combo box is editable) or you can load all names by pressing the *Load keys* button and then select one from the drop-down list.<sup>[1]</sup>

If you don't fill the *Key alias* field the first alias read from keystore will be used.

Each key in the keystore can be protected with its password. If this password differs from the password of keystore, fill it in the *Key password* input field.

## Signing properties

*JavaFX panel: Signature Properties*

### Input and output PDF files

*Input PDF file* is the existing PDF file to which a digital signature will be added.

*Output PDF file* is the name of the resulting PDF file. If not specified, the *Input PDF file* name with the suffix **signed is used automatically** (e.g. `input_test.pdf` will produce `test_signed.pdf`)

*\*The Input and Output files have to be different!\**

## Reason, location, contact

The reason, location, and contact fields provide additional information about the signature. Filled values will be stored in the result PDF.

## Append signature

JSignPdf can work in two signing modes. When *Append signature* is enabled, the new signature is appended and any previously-existing signatures stay unchanged; when it is disabled, existing signatures are replaced. The GUI enables *Append signature* by default; on the CLI the default is replace (pass **-a** to append). *\*This option is disabled for encrypted documents.\**

## Certification level

The JSignPdf application can add a certificate to the signed PDF. There are four levels of certification:

- NOT\_CERTIFIED
- CERTIFIED\_NO\_CHANGES\_ALLOWED
- CERTIFIED\_FORM\_FILLING
- CERTIFIED\_FORM\_FILLING\_AND\_ANNOTATIONS

## Hash algorithms

You can choose which hash function will be used for the signature. Available algorithms include SHA-256, SHA-384, SHA-512, and others.

## Remember passwords

JSignPdf stores filled information when you are exiting the application, so it's present when you run it the next time. Passwords are stored (encrypted) when the *Remember passwords* option (labelled *Store passwords* in the JavaFX GUI) is enabled. The option is enabled by default; uncheck it if you prefer to re-enter passwords on every launch.

*\*Even if the password is stored in the encrypted form, we do not recommend storing passwords if your computer is used by more users!\**

# Encryption

*JavaFX panel: Encryption & Rights*

The *Encryption* option enables additional fields for support of PDF security. By using this you can either sign secured PDFs (and change the rights and user password) or you can add encryption to unencrypted PDF during the signing.

## Encryption: Passwords

Fill owner and user passwords to set it in secured result PDF. If the input PDF is encrypted, the *Owner password* field has to match to owner password of the input PDF.

## Encryption: Certificate

Fill the path to a certificate file (\*.cer, \*.crt, ...) which should be used for the PDF encryption. Only the user who has the private key for the certificate will be able to open the file.

## Rights

You can set allowed actions in encrypted result PDF by configuring the document rights. Normal rights are represented by checkboxes. Printing right has 3 levels, so the combo box is used for it.

## Visible signature options

*JavaFX panel: Signature Appearance*

Enabling the *Visible signature* option allows you to create a visible field with signature directly in the signed PDF. You can configure parameters (position/texts/images) of the visible signature.

The screenshot shows a JavaFX panel titled "Signature Appearance" with the following configuration options:

- Enable visible signature:** A checkbox that is checked.
- Render mode:** A dropdown menu set to "Image and description".
- Signature text:** A text input field with a blue border.
- Status text:** A text input field.
- Font size:** A text input field containing the value "10.0".
- Signature image:** A text input field with a folder icon button next to it.
- Background image:** A text input field containing the path "/home/kwart/Pictures/jsmith-sig.pn" with a folder icon button next to it.

At the bottom of the panel, there are two expandable sections: "Timestamp & Validation" and "Encryption & Rights".



## Page

Page number (counted from 1) to which the signature will be added.

## Signature corners

The four inputs *Lower Left (X, Y)* and *Upper Right (X, Y)* define the position of the signature on the page. You can fill in float numbers (with decimal places) as input. Negative values are subtracted from the page width (for X) or height (for Y), allowing you to position the signature relative to the right or top edge.

The zero ([0,0]) position on the page is in the left bottom corner.

In the JavaFX interface you can place the signature visually by holding Shift and dragging a rectangle on the document preview.

## Display

In the *Display / Render mode* option you can set which fields will be generated to visible signature.

## Acrobat 6 layers

The *Acrobat 6 layer mode* option (enabled by default) allows you to control which signature layers will be added to the signed document. Acrobat 6.0 and higher recommends that only layers n2 and n4 be present. If the option is disabled then all layers will be created.

## Texts and Images

*Signature Text*, *Status Text*, *Image*, and *Background Image* inputs define the content of fields in a visible signature. *Font Size* is used for setting the size of *Signature Text*, it should contain a positive decimal number.

*Background image scale* defines the size of a background image. Any negative number means the best-fit algorithm will be used. Zero value means to stretch, which fills the whole field — it doesn't keep the image ratio. A positive value means the multiplicator of the original size.

Supported file formats for the *Image* and *Background Image* are GIF, JPEG, JPEG2000, PNG, WMF, BMP, and TIFF.

## TSA — timestamps

*JavaFX panel: Timestamp & Validation*

To add a timestamp to the signature you will need a timestamping authority (TSA). Fill the server address into the *TSA URL* field and if the server requires authentication, choose the authentication type and fill either the *TSA User* and *TSA Password* fields or the path to the certificate's private key (it has to be a PKCS#12 keystore) and the password. You can also set *TSA Policy OID*, which will be sent to the TSA server in the request, but typically the server uses the right policy by itself.

# Certificate revocation checking

*JavaFX panel: Timestamp & Validation*

JSigntPdf supports two standard ways of certificate revocation checking — CRL and OCSP. Most X.509 certificates support CRL, but it has some disadvantages (for instance the size of the list and possibly outdated information). OCSP solves these issues, but not all Certification Authorities (CA) support it.

## CRL

A Certificate Revocation List (CRL) is a list of serial numbers for certificates that have been revoked or are no longer valid (RFC 5280). When CRL checking is enabled in JSigntPdf, the revocation list is downloaded from the CA and embedded in the signed PDF.

## OCSP

The Online Certificate Status Protocol (OCSP) allows real-time verification of a certificate's revocation status (RFC 6960). Unlike CRL, which requires downloading a potentially large list, OCSP queries the CA for the status of a single certificate.

If OCSP is enabled in JSigntPdf and the protocol is supported for the certificate, the OCSP request will be created and the response will be stored in the signed PDF. The URL of the OCSP server is retrieved from the certificate. If the OCSP part is not found in the signing certificate, the value from the *default OCSP server URL* field will be used.

## Proxy settings

If some "online" feature (TSA, CRL, OCSP) is enabled and JSigntPdf runs behind a firewall, you can set the proxy, which will be used for all internet connections. Proxy type DIRECT means no proxy will be used.

---

[1] Only the private keys, which are valid (at the time of the signing) are displayed in the list. If the certificate supports the Key Usage extension, the private key will only be displayed if it is meant for signing.

# Using hardware tokens for signing

Steps to sign documents using hardware tokens:

1. Install PKCS#11 driver for your token. Check the vendor's documentation and install a proper driver for your system;
2. Place a configuration file at `<config-dir>/pkcs11.cfg` (see [Configuration directory](#)). It is used to configure a Java SunPKCS11 security provider (see [PKCS#11 Reference Guide](#)). The easiest way to create or edit it is via *File > Preferences > PKCS#11* in the JavaFX UI.

The content depends on your driver, you can try to start with a simple 2 lines:

```
name=Test
library=/path/to/your/PKCSDriver.so
```

1. Try to run JSignPdf with PKCS11 debug enabled:

```
java -Djava.security.debug=pkcs11keystore \
-Djava.security.debug=sunpkcs11 \
-jar JSignPdf.jar
```

If it doesn't work, try to add parameter `slot` or `slotListIndex` into `pkcs11.cfg` file, e.g.:

```
name=Test
library=/path/to/your/PKCSDriver.so
slot=2
```

or

```
name=Test
library=/path/to/your/PKCSDriver.so
slotListIndex=1
```

Value used for `slot` or `slotListIndex` depends on how many certificates you have installed.

If the PKCS11 keystore type works properly in the GUI and you can use the certificate on your token, you're ready to use it also in the batch mode.

```
java -jar JSignPdf.jar -kst PKCS11 -ksp 123456 document.pdf
```

# Advanced application configuration

Application-wide tweaks beyond the per-document signing options live in `<config-dir>/advanced.properties` and — for hardware tokens — `<config-dir>/pkcs11.cfg`. Both are plain text files; the easiest way to edit them is the JavaFX [Preferences dialog](#), which writes them on *OK*. Power users can also hand-edit the files directly while JSignPdf is closed; the next launch picks up the changes.

`advanced.properties` covers the same six topics as the Preferences dialog (visible-signature font, certificate-validation toggles, relaxed SSL, PDF preview backend order, default TSA hash algorithm, and one or two miscellaneous knobs). When a key is missing from `advanced.properties`, JSignPdf falls back to defaults bundled inside `JSignPdf.jar`, so a fresh install needs no config file at all.

## Migration from earlier versions

In JSignPdf 2.x the same options lived in `<install dir>/conf/conf.properties` and the PKCS#11 provider was pointed to by a `pkcs11config.path` key in that file. Starting with 3.0.0 those files move under your per-user [configuration directory](#):

- `<install dir>/conf/conf.properties` → `<config-dir>/advanced.properties`
- `<install dir>/conf/pkcs11.cfg` → `<config-dir>/pkcs11.cfg` (a fixed well-known location; the `pkcs11config.path` key has been removed)

The 3.0.0 distribution no longer ships an `<install dir>/conf/` directory and there is no automatic migration of pre-3.0.0 install-dir tweaks. If you previously customized `conf.properties` or maintained a `pkcs11.cfg`, re-enter the values via the Preferences dialog — or copy your old files into `<config-dir>/advanced.properties` and `<config-dir>/pkcs11.cfg` by hand before launching 3.0.0.

## Configuration directory

Starting with JSignPdf 3.0.0, per-user state is stored in a platform-native configuration directory instead of a single file in your home folder. The directory holds the main config file (`config.properties`, which persists the last-used signing settings), a `presets/` subdirectory (see [Using presets](#)), and — once you save changes from the Preferences dialog — the advanced-config files `advanced.properties` and `pkcs11.cfg` (see [Advanced application configuration](#)).

The default location depends on your operating system:

| Platform    | Default location                                                                                          |
|-------------|-----------------------------------------------------------------------------------------------------------|
| Linux / BSD | <code>\$XDG_CONFIG_HOME/jsignpdf/</code> , falling back to <code>\$HOME/.config/jsignpdf/</code>          |
| Windows     | <code>%APPDATA%\JSignPdf\</code> (typically <code>C:\Users\&lt;you&gt;\AppData\Roaming\JSignPdf\</code> ) |
| macOS       | <code>~/Library/Application Support/JSignPdf/</code>                                                      |

You can override the default by setting the `JSIGNPDF_CONFIG_DIR` environment variable to any directory path — useful for portable installs, scripted tests, or keeping multiple profiles side-by-side.

## Layout

```
<config-dir>/
  config.properties          # main configuration (last-used signing settings)
  advanced.properties        # advanced preferences (created on first save)
  pkcs11.cfg                 # SunPKCS11 provider config (optional)
  presets/
    preset-1745270400123.properties
    preset-1745270500456.properties
    ...
```

Preset filenames are opaque (`preset-<epoch-millis>.properties`); the user-visible name lives inside the file as `preset.displayName`. You can read, copy, or hand-edit any of these files—they are plain Java properties.

## Migration from earlier versions

Previous versions of JSignPdf stored the main settings in `~/.JSignPdf` (a single file in your home directory). On the first run of JSignPdf 3.0.0 or newer, if the new configuration directory does not yet exist, JSignPdf copies `~/.JSignPdf` to `<config-dir>/config.properties` and creates an empty `presets/` subdirectory. **The legacy file is left in place**, so if you downgrade to an older JSignPdf release, it continues to work with its old settings. Pre-3.0.0 install-dir tweaks (`conf/conf.properties` and `conf/pkcs11.cfg`) are **not** migrated automatically—see [Advanced application configuration](#) for what to do.

Once the new directory exists, the legacy locations are no longer consulted or updated.

## Java VM options for the Windows EXE launchers

If you need to change JVM properties for the Windows launchers (for example to increase the maximum heap), edit the corresponding `.cfg` file under the `app` subfolder of the installation. jpackage creates one `.cfg` per launcher:

| Launcher           | Configuration file     |
|--------------------|------------------------|
| JSignPdf.exe       | app\JSignPdf.cfg       |
| JSignPdf-swing.exe | app\JSignPdf-swing.cfg |
| JSignPdfC.exe      | app\JSignPdfC.cfg      |
| InstallCert.exe    | app\InstallCert.cfg    |

Each `.cfg` file contains a `[JavaOptions]` section with one `java-options=` entry per option. Add additional options as further `java-options=` lines, for example:

```
[JavaOptions]
java-options=--add-exports=java.base/sun.security.util=ALL-UNNAMED
java-options=-Xmx1g
java-options=-Dswing.aatext=true
```



The configuration files are shared between the installed build (%ProgramFiles%\JSigndf\app\)) and the portable ZIP (JSigndf\app\ inside the extracted archive).

# Troubleshooting

## Out of memory error

If you see an `OutOfMemoryError` in the program console, you need to allow Java to use more memory.

When running from the platform-independent ZIP, add `-Xmx<size>` to your Java command. The following example allows Java to use 512 MB of heap:

```
$ java -Xmx512m -jar JSignPdf.jar
```

When running from the Windows EXE / MSI / portable ZIP, add a new `java-options=-Xmx512m` line to the `.cfg` file of the affected launcher (see [Java VM options for the Windows EXE launchers](#)).

## SSL/TLS certificate errors

When JSignPdf connects to a server through HTTPS (e.g. to a TSA server for timestamping), it may fail with a `SSLHandshakeException`. This happens because Java ships with a keystore of trusted root certificates (`cacerts`), and if the server's certificate is not signed by one of these trusted authorities, the connection is refused.

If you trust the server, you can add its certificate to the Java `cacerts` keystore using the bundled `InstallCert` tool (see [InstallCert Tool](#)).

## Certificate not found in keystore

If JSignPdf doesn't list your certificate when you press *Load keys*:

- Verify the keystore file path and password are correct.
- Check that the certificate is still valid (not expired).
- If the certificate has a Key Usage extension, make sure it includes digital signature capability.

## Timestamp server unreachable

If timestamping fails:

- Verify the TSA URL is correct and the server is accessible.
- If you are behind a proxy, configure the proxy settings (see [Proxy settings](#)).
- Check if the TSA server requires authentication and configure it accordingly.
- If you get an `SSLHandshakeException`, see [SSL/TLS certificate errors](#).

# Other command line tools

## InstallCert Tool

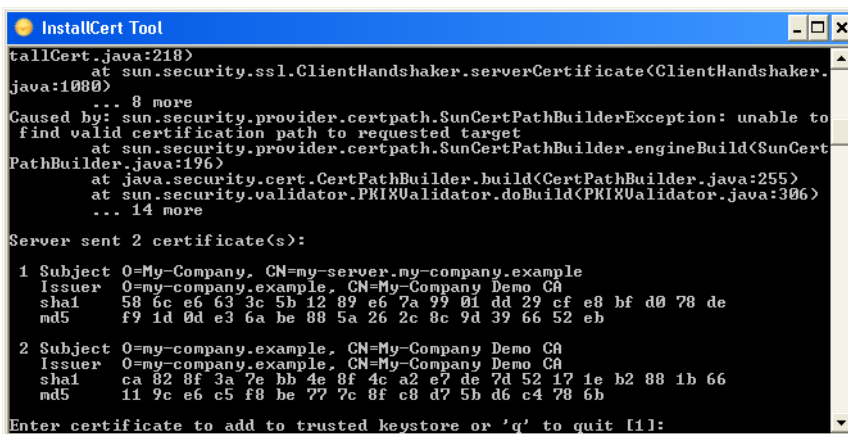
JSignPdf comes with a command line utility for adding server certificates to the Java `cacerts` keystore. This is useful when connecting to HTTPS servers whose certificates are not signed by a well-known CA (see [SSL/TLS certificate errors](#)).

```
$ java -jar InstallCert.jar
```

or

```
$ java -jar InstallCert.jar hostname[:port] [cacertPwd]
```

If you don't provide a hostname argument, you will be asked for it.



```
InstallCert Tool
tallCert.java:218)
    at sun.security.ssl.ClientHandshaker.serverCertificate(ClientHandshaker.
java:1088)
    ... 8 more
Caused by: sun.security.provider.certpath.SunCertPathBuilderException: unable to
find valid certification path to requested target
    at sun.security.provider.certpath.SunCertPathBuilder.engineBuild(SunCert
PathBuilder.java:196)
    at java.security.cert.CertPathBuilder.build(CertPathBuilder.java:255)
    at sun.security.validator.PKIXValidator.doBuild(PKIXValidator.java:306)
    ... 14 more
Server sent 2 certificate(s):
1 Subject: O=My-Company, CN=my-server.my-company.example
   Issuer: O=my-company.example, CN=My-Company Demo CA
   sha1  58 6c e6 63 3c 5b 12 89 e6 7a 99 01 dd 29 cf e8 bf d0 78 de
   md5   f9 1d 0d e3 6a be 88 5a 26 2c 8c 9d 39 66 52 eb
2 Subject: O=my-company.example, CN=My-Company Demo CA
   Issuer: O=my-company.example, CN=My-Company Demo CA
   sha1  ca 82 8f 3a 7e bb 4e 8f 4c a2 e7 de 7d 52 17 1e b2 88 1b 66
   md5   11 9c e6 c5 f8 be 77 7c 8f c8 d7 5b d6 c4 78 6b
Enter certificate to add to trusted keystore or 'q' to quit [1]:
```

The certificate chain will be displayed and you can choose which one will be imported.